Sunthareswaran Sharujan

TD05 & TD06 : Projet MarketPulse

Branches distantes, Pull Request et code review

Module MESIFI472326 – Python, Git, Linux · Groupe G07 · Équipe TSHA



Enseignant : ELANGA NKO'O Steve Patrick Lionel **Année Universitaire :2026/2027**

Table des matières

1. Introduction **3**

2. TD05 – Remote Branch + Pull Request **3**

2.1. Base commune de l'équipe 3
2.2. Parties 1 et 2 – Local, distant et répartition des tâches 4
2.3. Partie 3 – Création de la branche 4
2.4. Partie 4 – Implémentation 4
2.5. Partie 5 – Commit 5
2.6. Partie 6 – Push de la branche 6
2.7. Parties 7 et 8 – Pull Request 6
2.8. Partie 10 – Statut de la PR 6

3. TD06 – Code Review **7**

3.1. Review reçue sur ma PR 7
3.2. Ma review sur la PR d'un coéquipier 7
3.3. Mise à jour de main après le merge 7

4. Conclusion **7**

1. Introduction

Jusqu'au TD04, chacun travaillait en local dans son Codespace. Les TD05 et TD06 ajoutent le travail à plusieurs sur le dépôt de l'équipe (Henri127/esilv-marketpulse-starter-g07-tSHA) : envoyer sa branche sur GitHub, ouvrir une Pull Request, la faire relire par un coéquipier, puis l'intégrer dans `main`. Dans l'équipe, Henri a publié le `main` commun, et Adrien et moi avons chacun travaillé sur notre propre branche.

2. TD05 – Remote Branch + Pull Request

2.1. Base commune de l'équipe

Après les TD03 et TD04, chacun avait un `main` local différent. Henri a publié le `main` de référence sur GitHub, puis je me suis aligné dessus :

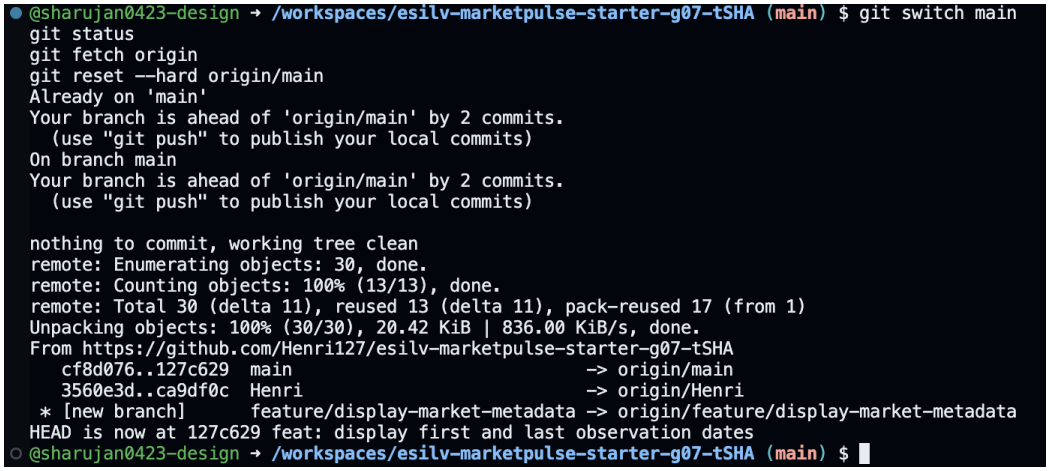


Figure 1 – git fetch origin puis git reset --hard origin/main.

`git fetch` récupère les nouveautés du dépôt distant sans toucher à mes fichiers, puis `git reset --hard origin/main` remet mon `main` exactement sur celui de l'équipe (commit `127c629`). Cette commande efface les commits locaux de `main`, c'est pour ça que l'énoncé ne l'autorise qu'à ce moment précis, avec un dépôt propre.

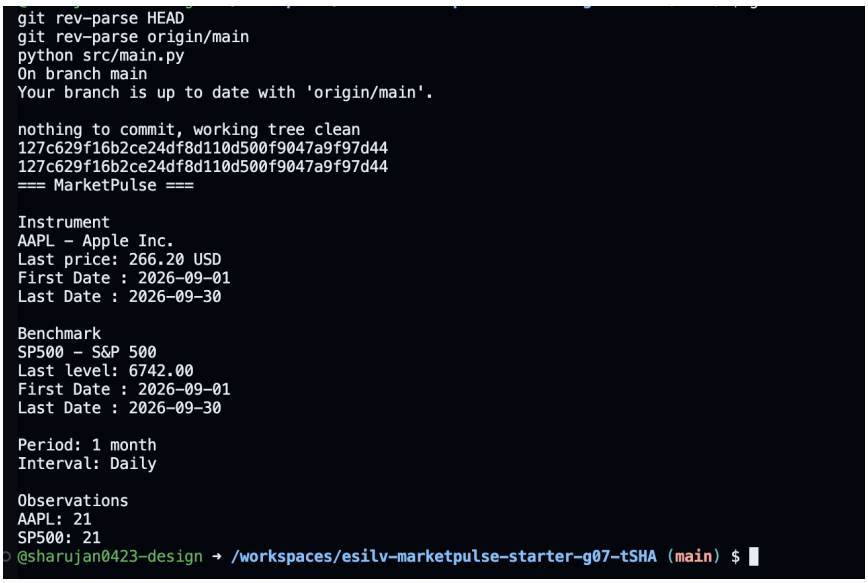


Figure 2 – HEAD et origin/main identiques, et MarketPulse fonctionne.

Les deux `git rev-parse` renvoient le même identifiant : mon `main` local est bien identique à `origin/main`, et le programme fonctionne sur cette base.

2.2. Parties 1 et 2 – Local, distant et répartition des tâches

Une branche locale n'existe que dans mon environnement. Après un `git push`, elle existe aussi sur GitHub sous le nom `origin/<branche>`. `origin` désigne le dépôt de l'équipe, d'où vient mon Codespace.

Pour ne pas coder deux fois la même chose, on s'est réparti les tâches. J'ai pris l'affichage du fournisseur de données (provider). Une branche `feature/display-market-metadata` existait déjà sur GitHub pour une autre tâche.

2.3. Partie 3 – Création de la branche



Figure 3 – Création de feature/display-provider-label depuis main.

J'avais d'abord créé une branche `feature/display-market-metadata` en local, que j'ai supprimée sans y avoir rien commité (elle était encore sur `127c629`), pour repartir sur un nom qui correspond à ma tâche : `feature/display-provider-label`.

2.4. Partie 4 – Implémentation

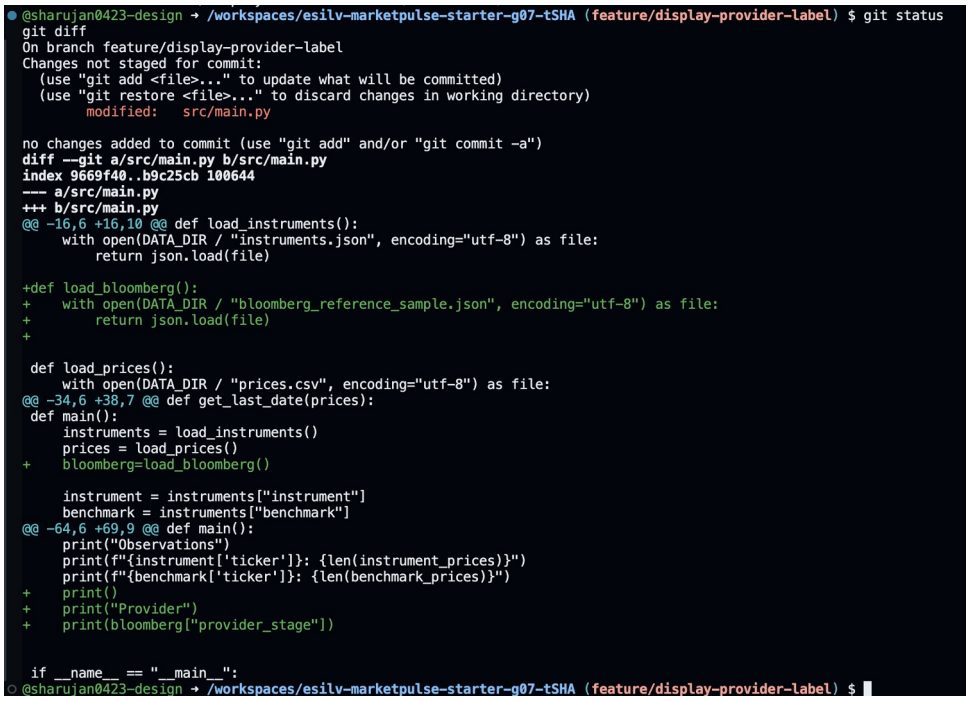


Figure 4 – git status et git diff sur la branche.

J'ai ajouté une fonction `load_bloomberg()` qui lit `data/sample/bloomberg_reference_sample.json`, et j'affiche à la fin du programme le champ `provider_stage` de ce fichier sous le titre « Provider ». Seul `src/main.py` est modifié.

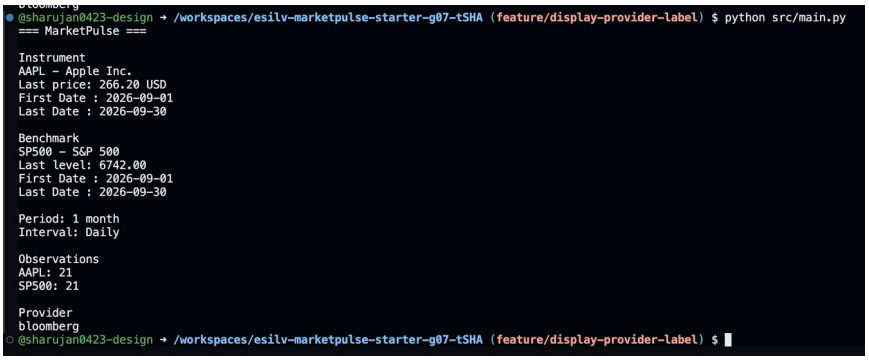


Figure 5 – python src/main.py : la ligne Provider apparaît à la fin.

Le programme fonctionne toujours et affiche le fournisseur (« bloomberg »). Les prix affichés viennent toujours du CSV : la ligne indique l'étape de fournisseur prévue dans le fichier de référence.

2.5. Partie 5 – Commit

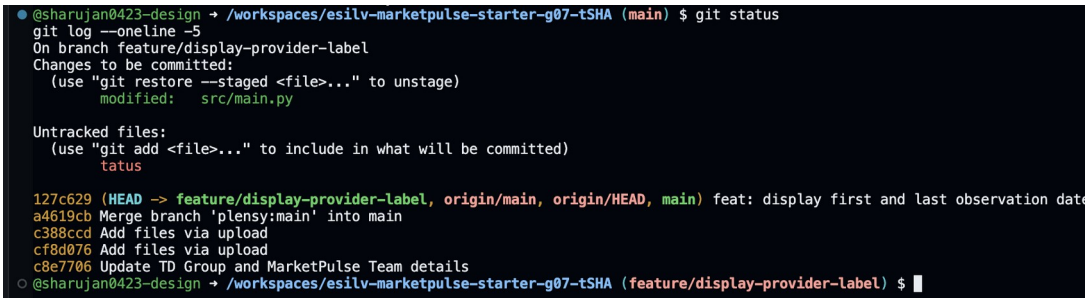


Figure 6 – État avant le commit : src/main.py indexé.

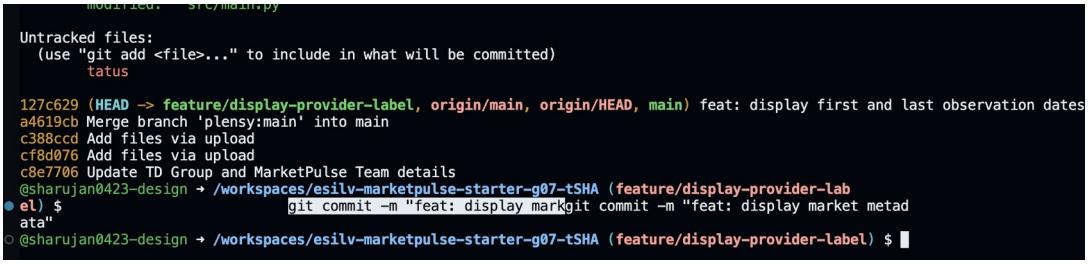


Figure 7 – git commit sur la branche.

Mon premier commit a été créé avec le message `feat: display market metadata`, alors que le changement concerne le fournisseur : j'ai repris par erreur le nom de la tâche initiale. Le commit suivant de la branche porte le bon message, `feat: display provider label`. On voit aussi un fichier parasite `tatus`, créé par erreur dans l'affichage de `git log` (touche s) et jamais commité.

2.6. Partie 6 – Push de la branche

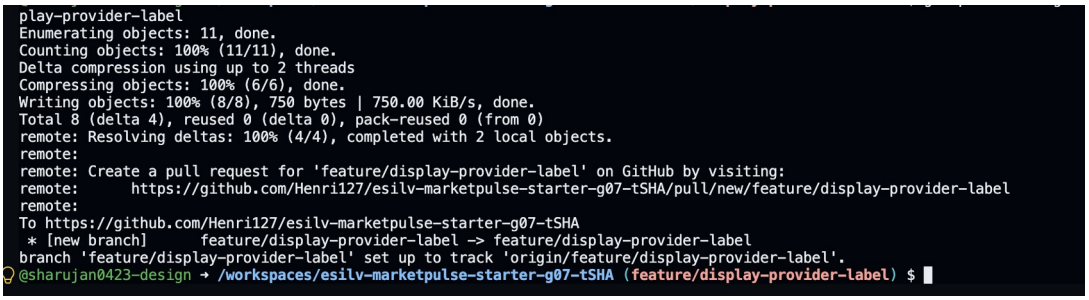


Figure 8 – git push -u origin feature/display-provider-label.

La branche est envoyée sur GitHub (`[new branch]`). L'option `-u` relie ma branche locale à `origin/feature/display-provider-label` : les prochains `git push` depuis cette branche n'auront plus besoin de préciser la destination. GitHub propose directement le lien pour créer la Pull Request.

2.7. Parties 7 et 8 – Pull Request

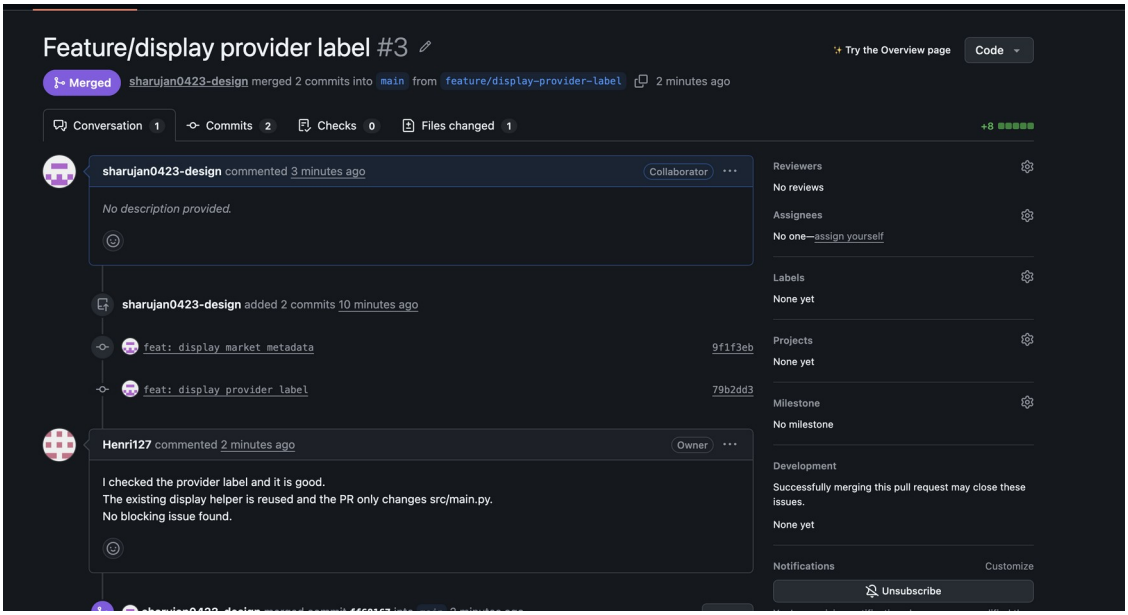


Figure 9 – Pull Request #3 sur le dépôt de l'équipe.

La PR #3 va de `feature/display-provider-label` (source) vers `main` (cible), dans le dépôt de l'équipe et pas dans celui du prof. Elle contient mes 2 commits et un seul fichier modifié (+8 lignes), ce qui correspond bien au travail prévu.

Deux points auraient dû être mieux faits : le titre a été généré à partir du nom de la branche (« Feature/display provider label ») au lieu d'un titre du type `feat: display provider label`, et je n'ai pas écrit de description. Elle aurait dû préciser ce qui change (affichage du fournisseur), pourquoi (donner le contexte de la source des données) et comment c'est validé (`python src/main.py`, un seul fichier modifié).

2.8. Partie 10 – Statut de la PR

L'énoncé demande de laisser la PR ouverte pour la review du TD06. Dans notre cas, Henri l'a relue juste après son ouverture et je l'ai fusionnée ensuite : la relecture a donc bien eu lieu avant le merge, mais plus tôt que prévu. Le TD06 décrit cette relecture.

3. TD06 – Code Review

3.1. Review reçue sur ma PR

Henri a relu ma PR et a laissé ce commentaire (figure 9) : « I checked the provider label and it is good. The existing display helper is reused and the PR only changes src/main.py. No blocking issue found. » C'est une review utile parce qu'elle dit ce qui a été vérifié (le label, le fichier modifié) et qu'elle conclut qu'il n'y a pas de problème bloquant. Aucune correction n'a donc été demandée, et la PR a été fusionnée dans `main` (statut « Merged », 2 commits).

Une review peut se terminer de trois façons : Comment (une remarque sans décision), Request changes (un vrai problème à corriger avant le merge) ou Approve (la PR est prête). Ici, la relecture a été faite sous forme de commentaire.

3.2. Ma review sur la PR d'un coéquipier

[À compléter : capture de ma review sur la PR d'Adrien ou d'Henri (Files changed + commentaire + décision), puis 2-3 lignes sur ce que j'ai vérifié.]

3.3. Mise à jour de main après le merge

[À compléter : capture de git switch main, git fetch origin, git reset --hard origin/main (ou git pull), puis git log --oneline -5 et python src/main.py.]

4. Conclusion

Ces deux TD m'ont fait passer du travail seul au travail en équipe. Une branche envoyée sur GitHub et une Pull Request rendent le travail visible et relisible par les autres avant d'arriver dans `main`. J'ai aussi vu l'importance des détails : un message de commit faux ou une PR sans description rendent la relecture plus difficile, même quand le code fonctionne.